

Universidade Federal do Maranhão
Centro de Ciências Exatas e Tecnologias
Engenharia da Computação

Bruno Feres de Souza

Disciplina: Métodos Formais

Código: EEC0019

17 de março de 2026

Preâmbulo

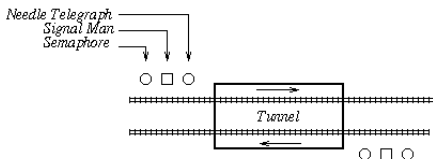
- Às vezes, o desenvolvimento de software pode ser difícil. Por exemplo:
 - Acidente no Túnel Clayton
 - Protocolo de autenticação
 - Problema da região crítica

Preâmbulo: Acidente no Túnel Clayton

- É um problema de protocolo.
- **Protocolos** são conjuntos de regras que governam a interação entre processos concorrentes em sistemas distribuídos.
 - Têm especificação muito simples.
 - Seu comportamento pode ser muito complexo (explosão combinatorial).
 - Certas ocorrências inesperadas requerem atenção imediata.

Preâmbulo: Acidente no Túnel Clayton

- O Túnel Clayton, em Brighton, Inglaterra, pode ser visualizado pelo modelo abaixo:



- Nele, há 3 sistemas para *garantir* a segurança do trânsito dos trens:
 - **Semáforos:** **verde** indica trilho disponível; **vermelho** indica trilho em uso. A transição de verde para vermelho deve ser automática. Caso haja problema na transição, um alerta sonoro é ativado.
 - **Sinaleiro:** responsável por alterar o semáforo de vermelho para verde, a fim de liberar, com segurança, o acesso para um novo trem.
 - **Telégrafo:** há três mensagens possíveis: *trem_no_tunel*, *tunel_livre* e *trem_saiu_do_tunel?*.

Preâmbulo: Acidente no Túnel Clayton

- O acidente ocorreu como segue:
 - O trem t_1 entra túnel, mas o semáforo não altera para vermelho. O alerta sonoro soou.
 - O sinaleiro s_1 transmite a mensagem *trem_no_tunel* pelo telégrafo e altera o semáforo para vermelho.
 - O trem t_2 se aproxima em alta velocidade, enquanto o semáforo ainda está verde e entra no túnel. Entretanto, o condutor do trem vislumbra a alteração do sinal e, tardiamente, pára o trem dentro do túnel.
 - O sinaleiro s_1 transmite novamente a mensagem *trem_no_tunel* pelo telégrafo. O protocolo não estabelece o significado de duas mensagens *trem_no_tunel* consecutivas. O problema consiste em como determinar que os dois trens saíram do túnel, deixando o túnel livre.

Preâmbulo: Acidente no Túnel Clayton

- O acidente ocorreu como segue:
 - Para determinar que o túnel está livre, o sinaleiro s_1 envia ao sinaleiro s_2 uma mensagem *trem_saiu_do_tunel_?*. Mesmo que o sinaleiro s_2 entenda a situação, ele não pode realizar a comunicação adequada. Então, quando o trem t_1 sai do túnel, o sinaleiro s_2 envia a mensagem *tunel_livre*.
 - O sinaleiro s_1 não sabe se ele deve esperar outro *tunel_livre* ou entender a mensagem literalmente. Ele decide por esta última opção e altera o semáforo para verde.
 - Como o condutor do trem t_2 vislumbrou o sinal vermelho no semáforo, ele decide, após algum tempo de deliberação, voltar à entrada.
 - Com o semáforo verde, o trem t_3 entra no túnel. O colisão dos trens t_2 e t_3 resultou em 21 mortes e 176 feridos.

Preâmbulo: Protocolo de autenticação

- Utiliza o **Protocolo Needham-Schröder**.
- A tarefa é como segue:
 - Os agentes **(A)**lice e **(B)**ob desejam trocar mensagens usando um canal de comunicação inseguro.
 - Depois da aplicação do protocolo, ambos os lados devem estar convencidos da autenticidade de seu parceiro.
 - Nenhum intruso deve ser capaz de decifrar as mensagens.

Preâmbulo: Protocolo de autenticação

- O protocolo opera da seguinte forma¹:
 - 1 A gera um código N_A e envia a mensagem $\langle A, N_A \rangle_B$ para B .
"Eu sou Alice e esta é minha mensagem."
 - 2 B gera um código N_B e envia a mensagem $\langle N_A, N_B \rangle_A$ para A .
"Eu sou Bob, já que eu consigo decifrar sua mensagem e aqui está a minha."
 - 3 A envia $\langle N_B \rangle_B$ para B .
"Eu sou Alice, já que eu posso decifrar a sua mensagem."
- Analisando o protocolo, tem-se que:
 - Depois do passo 2, A está convencido de estar tratando com B e aceita $\langle N_A, N_B \rangle$ como uma mensagem.
 - Depois do passo 3, B também está convencido de estar tratando com A e aceita $\langle N_A, N_B \rangle$ como uma mensagem.
- **Pergunta-se:** você confia neste protocolo?

¹ $\langle M \rangle_C$ significa que a mensagem M é encriptada usando a chave do agente C . O agente C pode gerar um código aleatório N_C .

Preâmbulo: Protocolo de autenticação

- Um intruso I pode:
 - Interceptar, ler, armazenar e responder mensagens.
 - Fazer parte do protocolo (com uma identidade falsa).
 - Utilizar apenas sua chave para decifrar mensagens.

Preâmbulo: Protocolo de autenticação

- O ataque do intruso I acontece da seguinte forma:
 - 1 A envia uma mensagem inicial para $\langle A, N_A \rangle_I$ para I (I se passa por B).
 - 2 I se fazendo passar por A envia a mensagem $\langle A, N_A \rangle_B$ para B .
 - 3 B envia a mensagem $\langle N_A, N_B \rangle_A$ para I .
 - 4 I encaminha essa mensagem para A .
 - 5 A envia $\langle N_B \rangle_I$ para I .
 - 6 I envia $\langle N_B \rangle_B$ para B .
- Analisando o ataque, tem-se que:
 - Depois do passo 1, I sabe a mensagem de A .
 - Depois do passo 5, I também sabe a mensagem de B .
 - Depois do passo 6, B está convencido de estar tratando com A .

Preâmbulo: Problema da região crítica

- O **problema da região crítica** consiste em proibir o acesso simultâneo de múltiplos processos a um recurso compartilhado.
- O que é desejável em um sistema concorrente?
 - 1 **Exclusão mútua**: apenas um processo por vez pode acessar o recurso.
 - 2 **Sem deadlock**: se vários processos tentarem acessar o recurso, um processo conseguirá.
 - 3 **Sem starvation**: se um processo tentar acessar o recurso, eventualmente ele conseguirá.

Preâmbulo: Problema da região crítica

- Tentativa 1:

```
turn : integer range 1..2=1;
task body P1 is
begin
  loop
    non_critical_section_1;
    loop exit when turn=1; end loop;
    critical_section_1;
    turn:=2;
  end loop;
end P1;

| task body P2 is
| begin
|   loop
|     non_critical_section_2;
|     loop exit when turn=2; end loop;
|     critical_section_2;
|     turn:=1;
|   end loop;
| end P2;
```

Preâmbulo: Problema da região crítica

- Sobre a tentativa 1:
 - **Exclusão mútua.** Considere que P1 e P2 estejam na região crítica. Suponha que P1 entrou primeiro. Se isso aconteceu, então inicialmente $\text{turn}=1$ e só depois $\text{turn}=2$. Como P1 ainda não atualizou turn , seria impossível que P2 já esteja na região crítica. Assim, por contradição, a hipótese de P1 e P2 estarem na região crítica ao mesmo tempo é falsa. Logo deve haver, de fato, exclusão mútua.
 - **Sem deadlock.** Sempre ou $\text{turn}=1$ ou $\text{turn}=2$. Logo, P1 ou P2 estará na região crítica.
 - **Com starvation.** Na ausência de competição, um processo pode esperar indefinidamente para acessar o recurso. Isso acontece, por exemplo, quando um processo termina (inesperadamente) antes de alterar o valor de turn .

Preâmbulo: Problema da região crítica

- Tentativa 2:

```
c1,c2 : integer range 0..1=1;
task body P1 is                                | task body P2 is
begin                                           | begin
  loop                                         |   loop
    non_critical_section_1;                   |     non_critical_section_2;
    c1:=0;                                    |     c2:=0;
    loop exit when c2=1; end loop;           |     loop exit when c1=1; end loop;
    critical_section_1;                      |     critical_section_2;
    c1:=1;                                    |     c2:=1;
    end loop;                                |     end loop;
end P1;                                        | end P2;
```

Preâmbulo: Problema da região crítica

- Sobre a tentativa 2:
 - **Com deadlock.** Para verificar a existência do problema, considere a execução dos seguintes passos:
 - 1 $(c1, c2) = (1, 1)$
 - 2 $c1 = 0$ em P1
 - 3 $c2 = 0$ em P2
 - 4 P1 testa $c2$ sem sucesso
 - 5 P2 testa $c1$ sem sucesso

Preâmbulo: Problema da região crítica

- Tentativa 3:

```
c1,c2 : integer range 0..1=1;
task body P1 is
begin
  loop
    non_critical_section_1;
    c1:=0;
  loop
    exit when c2=1;
    c1:=1;
    c1:=0;
  end loop;
  critical_section_1;
  c1:=1;
  end loop;
end P1;

| task body P2 is
| begin
|   loop
|     non_critical_section_2;
|     c2:=0;
|   loop
|     exit when c1=1;
|     c2:=1;
|     c2:=0;
|   end loop;
|   critical_section_2;
|   c2:=1;
|   end loop;
| end P2;
```


Preâmbulo: Problema da região crítica

- Sobre a tentativa 3:
 - **Com starvation.** Para verificar a existência do problema, considere a execução dos seguintes passos:
 - 1 $(c1, c2) = (1, 1)$
 - 2 $c1 = 0$ em P1
 - 3 $c2 = 0$ em P2
 - 4 P2 testa $c1$ e seta $c2 = 1$
 - 5 P1 faz o ciclo:
 - testa $c2$
 - vai para a região crítica
 - seta $c1 = 1$
 - vai para a região não-crítica
 - seta $c1 = 1$
 - 6 P2 seta $c2 = 0$
 - 7 vai para o passo 4

Preâmbulo: Problema da região crítica

- Tentativa 4 (**Algoritmo de Dekker**):

```
c1,c2 : integer range 0..1=1; turn : integer 1..2=1;
task body P1 is
begin
  loop
    non_critical_section_1;
    c1:=0;
    loop
      exit when c2=1;
      if turn=2 then c1:=1; loop exit when turn=1; end loop; c1:=0;
      end if;
    end loop;
    critical_section_1;
    c1:=1; turn:=2;
  end loop;
end P1;
```

Preâmbulo: Problema da região crítica

- Sobre a tentativa 4:
 - **Exclusão mútua.**
 - **Sem deadlock.**
 - **Com starvation.**
- Como provar?
 - utilizando **métodos formais.**

Bibliografia

- ① STRUTH, Georg. Notas de aula "Introduction to Formal Methods", da disciplina "Software Verification and Testing", do Departamento de Ciência da Computação da Universidade de Sheffield. 2008.
Disponível em:
<http://staffwww.dcs.shef.ac.uk/people/G.Struth/vt/l01.pdf>
- ② TOSCANI, S.S., OLIVEIRA, R.S., CARISSIMI, A.S. Sistemas Operacionais. Editora Bookman, Porto Alegre, 3ª Edição, 2008.
Disponível em:
<https://www.inf.ufrgs.br/asc/livro/cap4-14.pdf>
 - Capítulo 4.

Dúvidas?